Experiment No. 03 – Data Visualization

AIM: Data Visualization

Theory

What is the Purpose of Data Visualization?

Data visualization transforms raw numbers into graphical representations (charts, plots, maps) so that patterns, trends, outliers, and relationships in data become immediately apparent. It bridges the gap between complex datasets and human cognition, enabling faster and more accurate decision-making.

Benefits of Data Visualization Tools

- **Pattern Recognition** – Spot trends, clusters, and anomalies that are invisible in raw tables.
- **Effective Communication** – Convey complex findings to non-technical stakeholders quickly.
- **Faster Decision-Making** – Interactive dashboards allow real-time exploration.
- **Data Quality Insight** – Missing values, duplicates, and skewness become obvious.
- **Story-Telling** – Build compelling narratives around data.

Types of Visualization

Category	Techniques
Univariate	Histogram, Bar Chart, Quartile (Box) Plot, Distribution Chart
Multivariate	Scatter Plot, Scatter Matrix, Bubble Chart, Density Chart, Heatmap

Libraries Used

```
matplotlib · seaborn · plotly · bokeh
```

Dataset

Online Retail Dataset – Transactional data from a UK-based online retailer (541 909 rows × 8 columns).

Code Implementation

In [1]:

```
# ── Imports & Setup ──────────────────────────────────────────────────────────
import warnings, math
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import plotly.express as px
import plotly.graph_objects as go

from bokeh.plotting import figure, show
from bokeh.models import ColumnDataSource, HoverTool
from bokeh.transform import linear_cmap
from bokeh.palettes import Spectral6, Viridis256
from bokeh.io import output_notebook as bokeh_inline

bokeh_inline()           # Bokeh plots inside notebook
sns.set_theme(style="whitegrid")

# ── Load data ────────────────────────────────────────────────────────────────
df = pd.read_csv("data_online_retail.csv", encoding="latin1")
df.dropna(subset=["CustomerID"], inplace=True)
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])
df["TotalPrice"]  = df["Quantity"] * df["UnitPrice"]

# Keep only valid transactions (positive Quantity & UnitPrice)
df = df[(df["Quantity"] > 0) & (df["UnitPrice"] > 0)].copy()

print(f"Shape after cleaning: {df.shape}")
df.head()
```

Loading BokehJS ...

```
Shape after cleaning: (397884, 9)
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	Unit
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39
2	536365	84406B	CREAM CUPID	8	2010-12-01 08:26:00	2.75

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	Unit
			HEARTS COAT HANGER			
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39

Part A – Univariate Visualization

1. Histogram

Shows the frequency distribution of a single numeric variable.

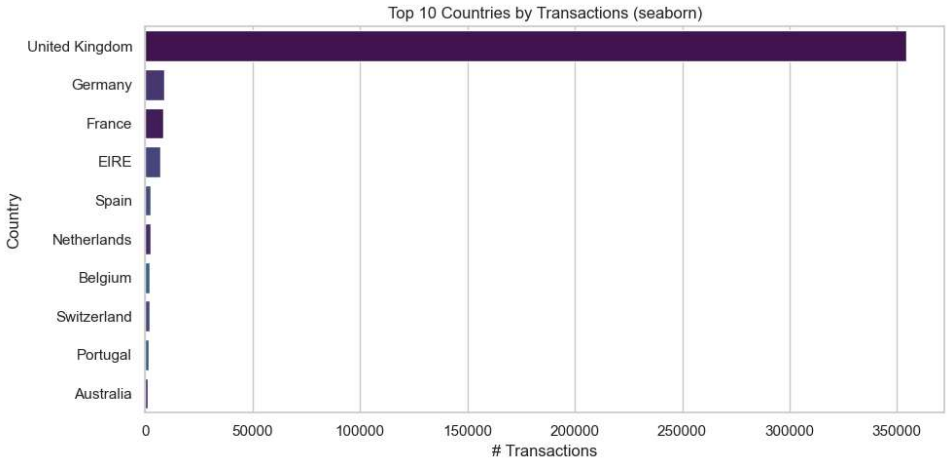
In [ ]:
```
# ── Histogram – Bokeh ──────────────────────────────────────────────
subset = df[df["UnitPrice"] < 50]["UnitPrice"]
hist_vals, edges = np.histogram(subset, bins=50)

p = figure(title="Histogram of Unit Price (bokeh)", width=700,
height=350,
           x_axis_label="Unit Price (£)", y_axis_label="Frequency")
p.quad(top=hist_vals, bottom=0, left=edges[:-1], right=edges[1:],
       fill_color="teal", line_color="white", alpha=0.8)
show(p)
```

2. Bar Chart

Displays the count / aggregate of a categorical variable.

In [7]:
```
# ── 2b  Bar Chart – Seaborn ──────────────────────────────────────────
fig, ax = plt.subplots(figsize=(10, 5))
sns.countplot(y="Country", data=df,
order=df["Country"].value_counts().head(10).index,
              hue="Country", palette="viridis", legend=False, ax=ax)
ax.set(title="Top 10 Countries by Transactions (seaborn)", xlabel="#
Transactions")
plt.tight_layout()
plt.show()
```



3. Quartile (Box) Plot

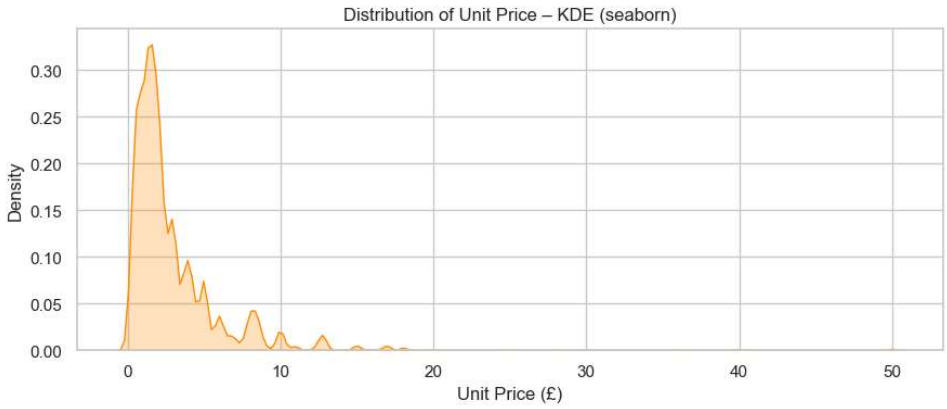
Shows the five-number summary (min, Q1, median, Q3, max) and outliers.

In [ ]:
```
# ──  Box Plot – Plotly ──────────────────────────────────────────────
top5 = df["Country"].value_counts().head(5).index
fig = px.box(df[df["Country"].isin(top5)], x="Country", y="UnitPrice",
             color="Country", title="Box Plot – Unit Price by Country
(plotly)")
fig.update_yaxes(range=[0, 30])
fig.show()
```

4. Distribution Chart (KDE / Violin)

Kernel Density Estimate (KDE) curves and violin plots show the probability distribution shape.

In [13]:
```
# ── 4a  KDE Distribution – Seaborn ──────────────────────────────────────
fig, ax = plt.subplots(figsize=(9, 4))
sns.kdeplot(df["UnitPrice"].clip(upper=50), fill=True,
color="darkorange", ax=ax)
ax.set(title="Distribution of Unit Price – KDE (seaborn)", xlabel="Unit
Price (£)")
plt.tight_layout()
plt.show()
```



In [14]:
```
# ── 4b  Violin Plot (Distribution) – Plotly ──────────────────────────────
top5 = df["Country"].value_counts().head(5).index
fig = px.violin(df[df["Country"].isin(top5)], x="Country",
y="UnitPrice",
                color="Country", box=True, points=False,
                title="Violin / Distribution Chart – Unit Price
(plotly)")
fig.update_yaxes(range=[0, 30])
fig.show()
```

Part B – Multivariate Visualization

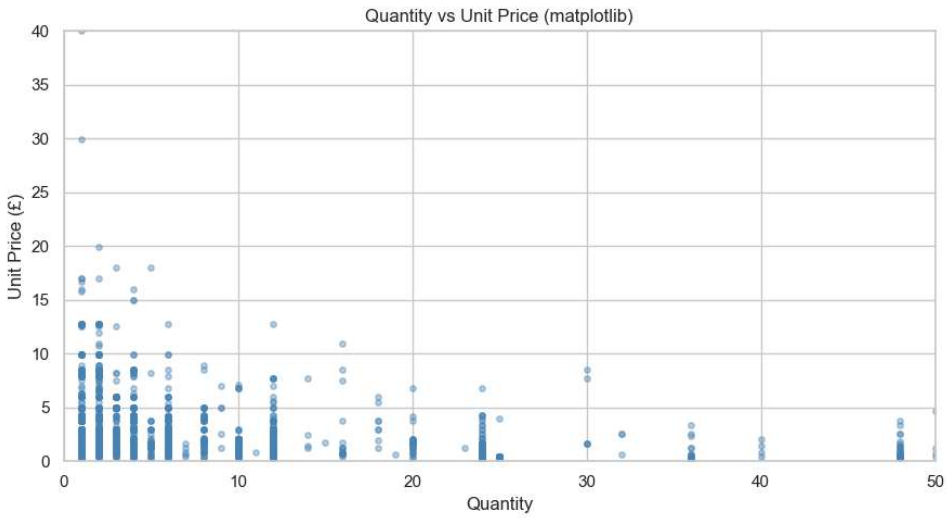
5. Scatter Plot

Shows the relationship between two numeric variables.

In [15]:
```
# ── 5a  Scatter Plot – Matplotlib
──────────────────────────────────────────────

sample = df.sample(2000, random_state=42)

fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(sample["Quantity"], sample["UnitPrice"], alpha=0.4, s=15,
c="steelblue")
ax.set(title="Quantity vs Unit Price (matplotlib)", xlabel="Quantity",
ylabel="Unit Price (£)",
       xlim=(0, 50), ylim=(0, 40))
plt.tight_layout()
plt.show()
```

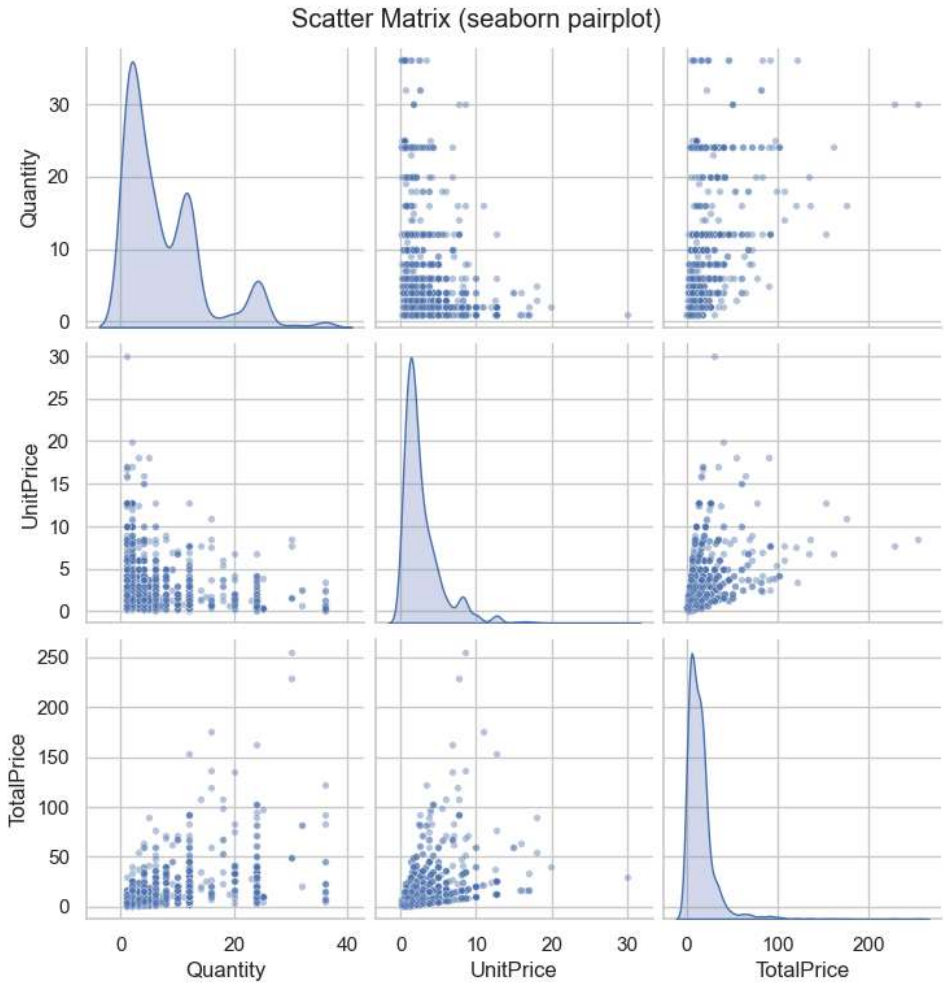


6. Scatter Matrix (Pair Plot)

A grid of scatter plots for every pair of numeric variables.

In [19]:
```
# ── 6a  Scatter Matrix – Seaborn (pairplot)
──────────────────────────────────────────
pair_df = sample[["Quantity", "UnitPrice", "TotalPrice"]].copy()
pair_df = pair_df[(pair_df["Quantity"] < 40) & (pair_df["UnitPrice"] <
40)]

g = sns.pairplot(pair_df, diag_kind="kde",
                 plot_kws={"alpha": 0.4, "s": 15},
                 diag_kws={"fill": True})
g.figure.suptitle("Scatter Matrix (seaborn pairplot)", y=1.02)
plt.show()
```



7. Bubble Chart

Like a scatter plot but a third variable is encoded as bubble size.

In [ ]:
```
# ── 7  Bubble Chart – Plotly ──────────────────────────────────────────
fig = px.scatter(country_agg, x="AvgQty", y="AvgPrice",
                 size="TotalRevenue", color="Country",
                 hover_name="Country", size_max=60,
                 title="Bubble Chart – Avg Qty vs Avg Price (plotly)")
fig.update_layout(showlegend=False)
fig.show()
```

8. Density Chart (2D KDE)

A 2-D kernel density estimate showing the joint distribution of two variables.

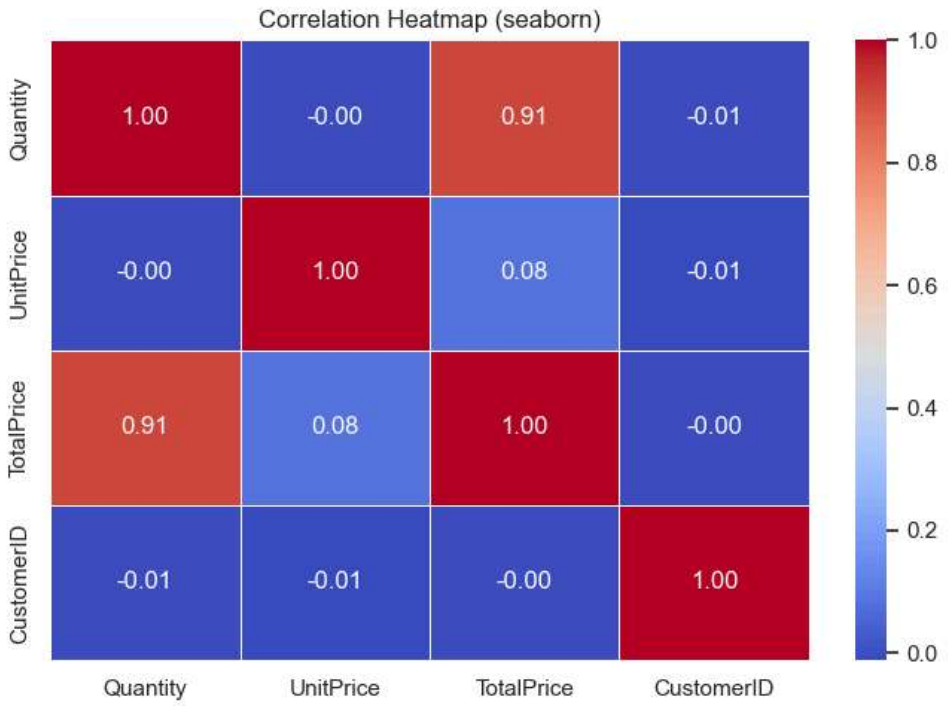
In [ ]:
```
# ── 8 Density – Plotly ────────────────────────────────────────────────
fig = px.density_contour(small, x="Quantity", y="UnitPrice",
                         title="2D Density Contour (plotly)")
fig.update_traces(contours_coloring="fill", colorscale="Blues")
fig.show()
```

9. Heat Map

Colour-coded matrix showing the correlation / magnitude between variables.

In [25]:
```
# ── 9a  Heatmap (Correlation) – Seaborn
──────────────────────────────────────
corr = df[["Quantity", "UnitPrice", "TotalPrice", "CustomerID"]].corr()

fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
linewidths=0.5, ax=ax)
ax.set_title("Correlation Heatmap (seaborn)")
plt.tight_layout()
plt.show()
```



Conclusion

In this experiment we demonstrated **9 visualization techniques** across **4 libraries** (matplotlib, seaborn, plotly, bokeh):

#	Technique	Type
1	Histogram	Univariate
2	Bar Chart	Univariate
3	Quartile (Box) Plot	Univariate
4	Distribution / KDE / Violin	Univariate
5	Scatter Plot	Multivariate
6	Scatter Matrix	Multivariate
7	Bubble Chart	Multivariate
8	Density Chart (2D KDE)	Multivariate
9	Heat Map	Multivariate

Each library has its strengths – **matplotlib** for fine control, **seaborn** for statistical aesthetics, **plotly** for interactivity, and **bokeh** for web-ready dashboards.

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.